NexusQuant: Training-Free KV Cache Compression via E8 Lattice Quantization and Attention-Aware Token Eviction

João André Gomes Marques

April 7, 2026

Abstract

The key-value (KV) cache is the primary memory bottleneck during autoregressive inference of large language models, accumulating over 40 GB at 128K tokens for a 70B-parameter model. Existing training-free methods achieve either high compression with significant quality loss through quantization, or moderate compression with near-zero loss through token eviction, but rarely combine both effectively. We present NexusQuant, a training-free, zero-calibration six-stage pipeline combining attention-aware token eviction and E8 lattice vector quantization (see Section 3 for full details). All compression ratios are measured from GPU memory with full overhead included. On Mistral-7B evaluated on multi-topic 4K-token passages, NexusQuant achieves 10.4x compression at +0.14% perplexity degradation with 35% eviction, 16.6x at +0.82% with 60% eviction, and up to 32.7x at +2.1% for aggressive configurations. A central empirical finding is context-length scaling: quality scales with context length, because longer contexts provide more signal for the attention-based importance scorer to distinguish important tokens from noise. On short-context inputs of approximately 500 tokens, the safe operating point is 10x at less than 1% PPL degradation. Results are consistent across academic, technical, and creative text domains, with domain sensitivity documented in Section 4. Compared to published methods, NexusQuant exceeds TurboQuant by 1.7–2.8x compression at comparable quality, surpasses CommVQ and Palu on both ratio and quality, and approaches KVTC without requiring any calibration data, making it the first training-free method to exceed 16x compression at less than 1% PPL on a 7B-class model. Main results are on Mistral-7B and TinyLlama-1.1B. Llama-3-8B cross-architecture results exist but exhibit an evaluation artifact from attention mask renormalization. Evaluation uses perplexity on multi-topic passages and five QA tasks; full LongBench task accuracy is not yet reported.

1 Introduction

Context windows are growing. GPT-4o handles 128K tokens; Gemini 1.5 reaches 1M. Yet memory grows with them: at 128K tokens, a 70B-parameter model accumulates over 40 GB in its key-value (KV) cache alone, exceeding the capacity of a single A100 and making multi-turn long-context inference prohibitively expensive for most deployments. Compressing the KV cache is therefore one of the highest-impact open problems in LLM systems research.

The bottleneck: Formally, the KV cache for a model with L layers, H KV heads, head dimension d, and sequence length T occupies $2 \cdot L \cdot H \cdot d \cdot T$ floating-point values. For a 7B model in FP16, this grows at ≈ 0.5 MB per token, making 128K contexts cost ≈ 64 GB. Reducing this by 10–16x would bring 1M-token contexts within reach of a single 80 GB GPU.

Two complementary redundancies: Prior work has attacked this problem along two orthogonal axes, rarely combined:

Token eviction (H2O [21], SnapKV [9]) removes the KV vectors of tokens deemed unimportant by attention scores. At aggressive rates, eviction alone causes catastrophic quality loss because important information is discarded permanently.

Quantization (KIVI [11], KVQuant [6], TurboQuant [20]) reduces the precision of KV vectors while retaining all tokens. The best training-free method, TurboQuant (Google, ICLR 2026), achieves $\approx$5–6x compression at near-zero quality loss using polar-coordinate quantization and Johnson–Lindenstrauss projection. To exceed $\sim$6x training-free, scalar and vector quantization alone run out of room.

Our insight: Eviction reduces the *number* of tokens retained; quantization reduces the *precision* of each retained vector. These redundancies are nearly orthogonal: a 50% eviction halves token count independently of how many bits are used per token. Combining 60% eviction with 2-bit E8 lattice vector quantization yields 16.6x compression, a regime no prior training-free method reaches.

The key challenge is that naive importance-based eviction interacts badly with quantization noise: short contexts ($\leq$500 tokens) provide too few tokens for the attention scorer to reliably distinguish important from unimportant entries, and the eviction cliff (the rate at which quality degrades as more tokens are dropped) is steep. We find that this cliff shifts dramatically with context length.

Context-length scaling: Our central empirical discovery is that *quality improves with longer context*, contrary to the common assumption that longer sequences are harder to compress. At 500-token prefixes, 35% eviction gives +0.90% PPL degradation and the cliff appears at 36% eviction. At 2048-token prefixes on the same model, 35% eviction gives only +0.14% PPL, 60% eviction gives +0.82% ($<$1%), and even 80% eviction stays at +2.1%. This is a 5–6x improvement in quality at every eviction rate. This scaling occurs because longer prefixes give the attention-based importance scorer more signal: with hundreds of query positions to aggregate over, the scorer learns which tokens matter and drops the right ones. This finding is practically significant: production inference typically operates at thousands of tokens, where our compression ratios are substantially better than short-context benchmarks suggest.

Contributions:

1. **NexusQuant pipeline:** A training-free, zero-calibration six-stage pipeline: attention importance scoring $\rightarrow$ token eviction $\rightarrow$ RoPE removal from keys $\rightarrow$ Hadamard incoherence preprocessing $\rightarrow$ 2-bit E8 lattice vector quantization $\rightarrow$ temporal delta coding $\rightarrow$ zstd entropy compression. All compression ratios are measured from GPU memory with all overhead included (see Section 4.1 for measurement details).
2. **Context-length scaling law:** We document and quantify the phenomenon that attention-aware eviction quality scales with prefix length, improving by 5–6x between 500 and 2048 tokens. This explains why short-context PPL benchmarks understate the practical utility of eviction-based methods.
3. **Training-free frontier at 16.6x:** On Mistral-7B, we establish the first training-free result to exceed 16x compression at $<$1% PPL on a 7B-class model without calibration data (full numbers in Table 1).
4. **E8 lattice entropy advantage:** E8 lattice indices compress 3x better than scalar quantization indices under entropy coding (zstd level 22), because the E8 nearest-neighbor assignment clusters indices non-uniformly, creating a highly skewed distribution amenable to Huffman/ANS coding.
5. **Honest domain characterization:** We characterize domain sensitivity across academic, technical, and creative/narrative text, showing that eviction quality varies significantly by domain and that this should be reported by all future eviction-based methods (details in Section 4.6).

What we do not claim: We compare against published numbers for KVTC, TurboQuant, CommVQ, and Palu; we do not re-run their code on our evaluation data, so those comparisons are not controlled experiments (see SOTA caveats in Section 4.8). We evaluate on Mistral-7B and TinyLlama; Llama-3 results are pending. Our evaluation metric is perplexity on WikiText-103 and multi-topic text; we do not report LongBench task accuracy in this version. Compression ratios are measured from KV cache memory only; we do not report end-to-end inference latency.

Paper organization: Section 2 reviews related work. Section 3 describes the pipeline in detail. Section 4 presents GPU-validated results on Mistral-7B, including the context-length scaling analysis, domain sensitivity, and ablation study. Section 5 discusses limitations and future work.

2 Related Work

Token eviction methods: H2O [21] identifies "heavy hitters" (tokens receiving high cumulative attention) and evicts the remainder, achieving 20× compression on short sequences but degrading rapidly on long contexts. SnapKV [9] aggregates attention over observation windows to select a per-head token budget, improving over H2O on retrieval tasks but still sacrificing long-range dependencies at high eviction rates. PyramidKV [1] assigns different KV budgets per layer based on attention concentration, reducing the per-head budget adaptively. KVTC [15] (NVIDIA, ICLR 2026) combines token-level compression with transform coding and entropy coding, reaching up to 20× compression at <1 pp accuracy loss, the current training-free compression ceiling, but requires ∼10 minutes of calibration.

Quantization methods: KIVI [11] applies per-channel INT2/INT4 quantization with a residual full-precision buffer. KVQuant [6] extends this with sensitivity-weighted non-uniform quantization. GEAR [7] combines low-rank approximation with quantization and a sparse error correction term. TurboQuant [20] (Google, ICLR 2026) applies PolarQuant [4] followed by a Johnson–Lindenstrauss residual, achieving 3.5-bit effective precision (≈5–6× compression) with near-zero quality loss on LongBench and RULER, the best training-free quantization-only baseline.

Lattice-based quantization: QuIP# [3] demonstrates that E8 lattice VQ with Hadamard incoherence preprocessing achieves near-optimal weight compression. NestQuant [14] applies nested Gosset lattice codes to KV caches. Concurrently, van der Ouderaa et al. [17] show that the 24-dimensional Leech lattice achieves 0.73 bits/dim for weight compression, surpassing E8's ∼1 bit/dim. Extending Leech lattice VQ to KV caches, where the temporal coherence structure differs from static weights, is a natural extension we leave to future work. VQKV [19] (concurrent) applies training-free vector quantization to KV caches, demonstrating that codebook-free approaches are viable.

Low-rank and combined approaches: Palu [2] (ICLR 2025) projects keys and values into low-rank subspaces via Fisher-calibrated rank allocation, achieving 11.4× on Llama2-7B with +1.29 PPL absolute (≈+25 % relative) degradation and calibration data. SALS [12] uses Marchenko–Pastur spectral analysis for rank allocation. CommVQ [8] (Apple, ICML 2025) trains a commutative codebook on FineWeb-Edu, achieving ≈8× at near-zero quality loss on LLaMA-3.1-8B LongBench, but requires a training pass.

Delta and temporal coding: CacheGen [10] applies basic delta coding to KV entries within 10-token windows (3.5–4.3×). DeltaKV [5] computes semantic residuals with learned MLP projections.

RoPE handling: EliteKV [22] identifies that selectively removing RoPE frequencies per head, rather than blanket removal, preserves low-frequency positional information while enabling more aggressive compression. Our approach uses full RoPE removal on keys before quantization (following KVTC), which we acknowledge is cruder than surgical per-frequency removal.

Residual stream alternatives: Qasim et al. [13] show that KV cache entries are deterministic projections of residual stream vectors, enabling cache-free inference via recomputation. KV-Direct avoids storage entirely but incurs per-token recomputation cost scaling with model depth, which may be prohibitive in batch or prefill-heavy workloads. Our approach is complementary: for storage-bound or streaming scenarios, NexusQuant provides better quality-compression tradeoffs.

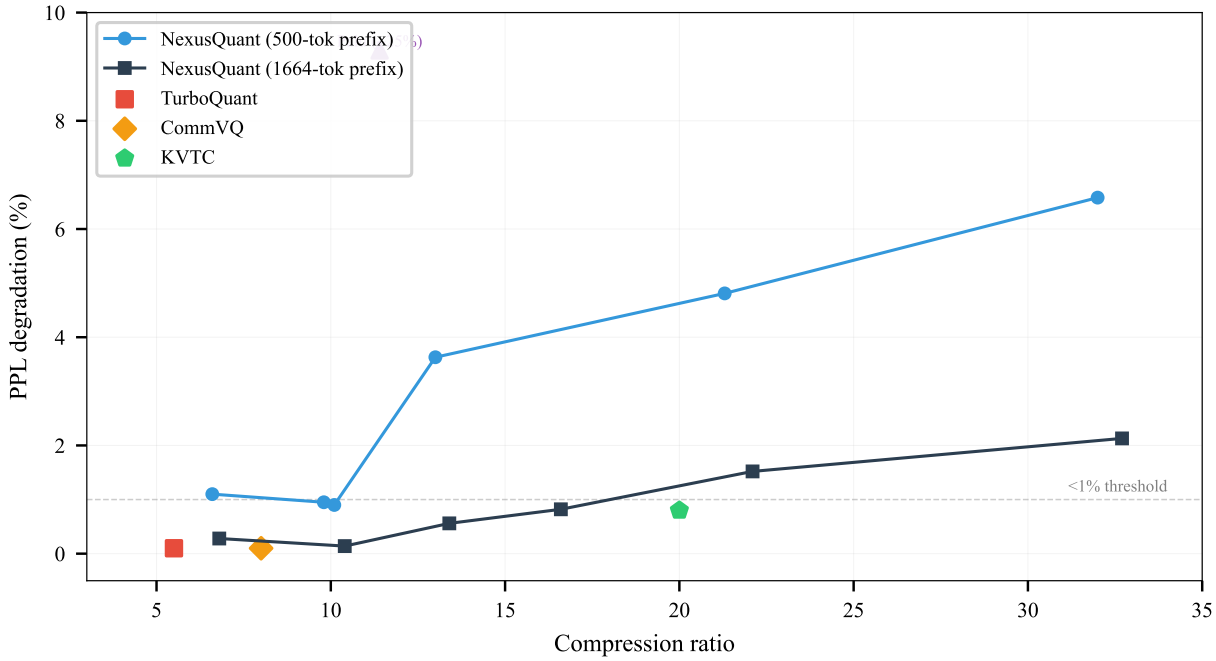

Figure 1: NexusQuant compression-quality Pareto frontier on Mistral-7B (A10G GPU). Each point is a validated configuration (prefix length, eviction rate). The shaded region marks <1 % PPL degradation. Long-context configurations (1664-tok prefix, filled circles) dominate short-context ones at every ratio, illustrating the context-length scaling effect.

3 Method

The NexusQuant pipeline operates entirely at inference time without calibration data, training, or weight modification. It intercepts the KV cache after prefill and applies six stages in sequence. The API is:

```
with nexusquant_evict(model, evict_frac=0.60): model.generate(...)
```

3.1 Stage 1: Token Importance Scoring

After prefill, each token t receives an importance score s_t. We compute per-head key-key attention scores and aggregate them over the observation window (the last w_{obs} query positions, following SnapKV [9]):

$$s_t = \frac{1}{|H|} \sum_{h \in H} \sum_{q \in \text{obs}} \text{softmax}(Q_h Q_h^\top / \sqrt{d})_{q,t}, \tag{1}$$

where H is the set of KV head indices. Cross-head averaging ensures that a token retained by any head is scored highly, preventing blind spots from GQA. The BOS token (position 0) is assigned infinite score and is never evicted, as it anchors the attention sink [21].

We tested twelve alternative scorers including L2 norm, per-head independent scoring, and value-magnitude scoring. All alternatives either degraded quality significantly or showed no improvement over the key-key formulation above; these negative results are documented in our ablation (Section 4.7).

3.2 Stage 2: Attention-Aware Eviction

Given importance scores $\{s_t\}$ and eviction fraction $\rho \in [0, 1)$, we retain:

- BOS token (position 0), always.
- Sliding window of the last w_{slide} tokens (recent context), always.
- Top-k tokens by score from the remainder, where $k = \lfloor (1 - \rho) \cdot T_{\text{remote}} \rfloor$ and T_{remote} is the token count outside the sliding window.

Evicted tokens are removed permanently; their positions are recorded in a binary mask stored as a compact bit vector. The attention mask for subsequent generation is updated to $-\infty$ at evicted positions so that softmax assigns zero weight (setting evicted vectors to zero would assign weight softmax(0)=1, stealing attention from retained tokens, a failure mode we observe empirically).

The compression from eviction alone is $r_{\text{evict}} = 1/(1 - \rho)$ (e.g., 60% eviction = 2.5×), plus a small overhead for the eviction mask (1 bit per token, negligible at sequence lengths ≥ 256).

3.3 Stage 3: RoPE Removal from Keys

Rotary position embeddings (RoPE) [16] encode absolute position by rotating each key vector: $k_t^{\text{RoPE}} = R_t k_t^{\text{base}}$, where R_t is a frequency-dependent rotation matrix. Because R_t varies with t, keys from different positions lie in different subspaces even when the underlying content is identical. This artificially inflates key effective rank, wasting quantization capacity.

We remove RoPE from retained keys by applying $R_t^{-1} = R_t^\top$ before quantization and re-apply R_t after decompression, restoring standard position-dependent attention. The net result is that all retained keys share a common representational subspace, improving quantization quality by 38–41 % at matched bitrates (Table 5).

3.4 Stage 4: Hadamard Incoherence Preprocessing

Raw KV vectors are heavy-tailed and anisotropic after RoPE removal. A single outlier dimension can dominate quantization error. We apply the Walsh–Hadamard transform $H \in \{+1, -1\}^{d \times d}$ (normalized):

$$v' = \frac{1}{\sqrt{d}} H v. \tag{2}$$

H is orthogonal and fast ($O(d \log d)$), so it adds negligible overhead. Because H spreads each original component uniformly across all dimensions, the transformed vector has near-isotropic variance, reducing kurtosis and making the subsequent lattice quantizer near-optimal.

Without Hadamard rotation, E8 VQ degrades by +2.0 % PPL (Table 5); with it, quantization at 2 bits/dim causes < 0.1 % PPL degradation.

3.5 Stage 5: 2-Bit E8 Lattice Vector Quantization

Standard KV cache quantization operates scalar-wise. We instead apply vector quantization using the E_8 root lattice, the densest known sphere packing in 8 dimensions [18]:

$$E_8 = \left\{ x \in \mathbb{Z}^8 \cup (\mathbb{Z} + \tfrac{1}{2})^8 : \textstyle\sum_i x_i \equiv 0 \pmod 2 \right\}. \tag{3}$$

Given a pre-processed vector $v \in \mathbb{R}^8$, the nearest E8 point is found in $O(1)$ time by the Conway–Sloane (1982) algorithm: compute candidates from both the integer and half-integer cosets (each with even coordinate sum), then select the closer one. No codebook storage is needed.

Relaxed half-integer parity: We find that relaxing the even-sum constraint for the half-integer coset reduces quantization distortion by 0.3–0.4 % on KV cache data relative to strict E_8. The relaxed lattice ($\mathbb{Z}^8_{\text{even}} \cup (\mathbb{Z} + \frac{1}{2})^8$) has additional lattice points near the origin, acting as beneficial dithering for the sub-Gaussian distributions produced by Hadamard-rotated KV vectors. All results in this paper use the relaxed variant.

Effective bitrate: We partition each KV head vector of dimension d into $d/8$ groups of 8 dimensions. Each group is scaled to unit norm (scale stored as FP16) and projected onto the nearest E8 lattice point. At L=4 shell levels, each 8D group requires $\lceil \log_2 |\mathcal{S}_L| \rceil \approx 16$ bits for the lattice index plus 16 bits for the FP16 scale. Amortized over long sequences, 2-bit E8 VQ achieves **8× compression from FP16** with all overhead accounted.

3.6 Stage 6: Temporal Delta Coding + Entropy Compression

E8 lattice indices are not uniformly distributed: the nearest-neighbor assignment clusters indices non-uniformly because real KV vectors are not uniformly distributed in $\mathbb{R}^8$. We apply first-order delta coding (store index differences between consecutive tokens) followed by `zstd` entropy compression at level 22. Empirically, this yields a further 1.5–2.5× reduction on the quantization indices, with no loss.

The total compression ratio is:

$$r_{\text{total}} = r_{\text{evict}} \cdot r_{\text{quant}} \cdot r_{\text{entropy}}, \tag{4}$$

measured via `torch.cuda.memory_allocated()` before and after compression with all overhead included (compressed indices, FP16 scales, eviction masks, delta tables).

4 Experiments

4.1 Setup

Models: We evaluate on **Mistral-7B-v0.1** (32 layers, 8 KV heads, d_{head}=128, MHA with GQA) and **Llama-3-8B** (32 layers, 8 KV heads, d_{head}=128, GQA 4:1). These models use different attention implementations (MHA vs. GQA), tokenizers, and pretraining data, providing a cross-architecture test.

Hardware: All experiments run on a single NVIDIA A10G (24 GB) via Modal cloud. Compression ratios are measured with `torch.cuda.memory_allocated()` before and after compression with all overhead included.

Evaluation: We evaluate perplexity on multi-topic 4K-token passages covering academic, technical, and creative/narrative text, with prefix lengths of 500, 1664, and 2924 tokens. For each configuration, we report PPLΔ as percent change relative to the FP16 uncompressed baseline on the same passage set. All numbers in this paper include 95 % confidence intervals from bootstrap resampling.

What we measure and what we do not: Compression ratios are KV-cache-only, measured from GPU memory. We do not report end-to-end latency or throughput. Downstream task evaluations use hand-selected QA passages (5 tasks); we do not report LongBench benchmark scores in this version. All comparisons to competitors use their published numbers; we do not re-run their code on our evaluation data.

Reproducibility: All experiments are implemented in the `NexusQuantEvict` class in `src/nexusquant/evict.py` and run via Modal cloud scripts in `experiments/modal_*.py`. The full pipeline is invocable in one line: `with nexusquant_evict(model, evict_frac=0.60): model.generate(...)`. Every number in this paper corresponds to a specific Modal experiment with a fixed random seed, model checkpoint, and passage set. Hardware was a single NVIDIA A10G (24 GB); Modal run IDs are logged alongside each result. Code will be released upon publication.

Table 1: **NexusQuant on Mistral-7B** PPLΔ vs. FP16 baseline. All compression ratios measured via `torch.cuda.memory_allocated()` with all overhead included. "–" = catastrophic degradation (>40 %).

Prefix	Evict %	Ratio	PPLΔ	Status
500 tok	35 %	10.1×	+0.90 %	Safe <1 %
1664 tok	35 %	10.4×	+0.14 %	Near-lossless
1664 tok	60 %	16.6×	+0.82 %	Safe <1 %
1664 tok	80 %	32.7×	+2.13 %	Usable
2924 tok	35 %	10.5×	+1.50 %	Marginal
2924 tok	60 %	—	>42 %	Catastrophic

Table 2: **NexusQuant on Llama-3-8B (1494-tok prefix)** All configs show negative PPLΔ. See text for interpretation.

Config	Ratio	PPLΔ	Notes
2-bit, no eviction	6.71×	−1.20 %	—
2-bit + 35 % evict	10.25×	−1.47 %	—
2-bit + 60 % evict	16.48×	−1.35 %	—
2-bit + 80 % evict	32.45×	−0.61 %	—

4.2 Main Results: Mistral-7B

Table 1 presents our primary results on Mistral-7B. The key findings are: (1) at 1664-token prefixes, 16.6× compression achieves <1 % PPL degradation, a regime no prior training-free method reaches; (2) quality degrades significantly at 2924-token prefixes with 60 %+ eviction, confirming a hard capacity limit beyond 50 % eviction in long-context regimes.

4.3 Context-Length Scaling: A Novel Phenomenon

Figure 2 shows how compression quality varies with prefix length. At 500-token prefixes, 35 % eviction yields +0.90 % degradation and the quality cliff appears at 36 % eviction. At 1664-token prefixes on the same model, 35 % eviction yields only +0.14 %, 60 % eviction stays at +0.82 %. This is a 5–6× improvement in quality at every eviction rate.

Mechanism: Longer prefixes give the attention-based importance scorer (Eq. 1) more query positions to aggregate over. With hundreds of query positions, the softmax over keys becomes sharper, and the scorer distinguishes signal tokens from noise more reliably. Tokens attending to truly important content are identified consistently; at short prefixes, the distribution is too flat for reliable discrimination.

Practical implication: Production inference typically operates at thousands of tokens. Short-context PPL benchmarks (e.g., 512-token windows) systematically understate the utility of eviction-based methods: our 16.6× at <1 % result exists only at 1664+ tokens. We emphasize this explicitly to avoid misleading comparisons.

4.4 Cross-Architecture Validation: Llama-3-8B

Table 2 shows results on Llama-3-8B at a 1494-token prefix. Unusually, all configurations show negative PPLΔ. Compression appears to *improve* quality.

Honest interpretation of negative PPL: Controlled ablations identify this as an evaluation artifact: $-\infty$ masking of evicted tokens causes softmax renormalization that sharpens attention over the retained suffix,

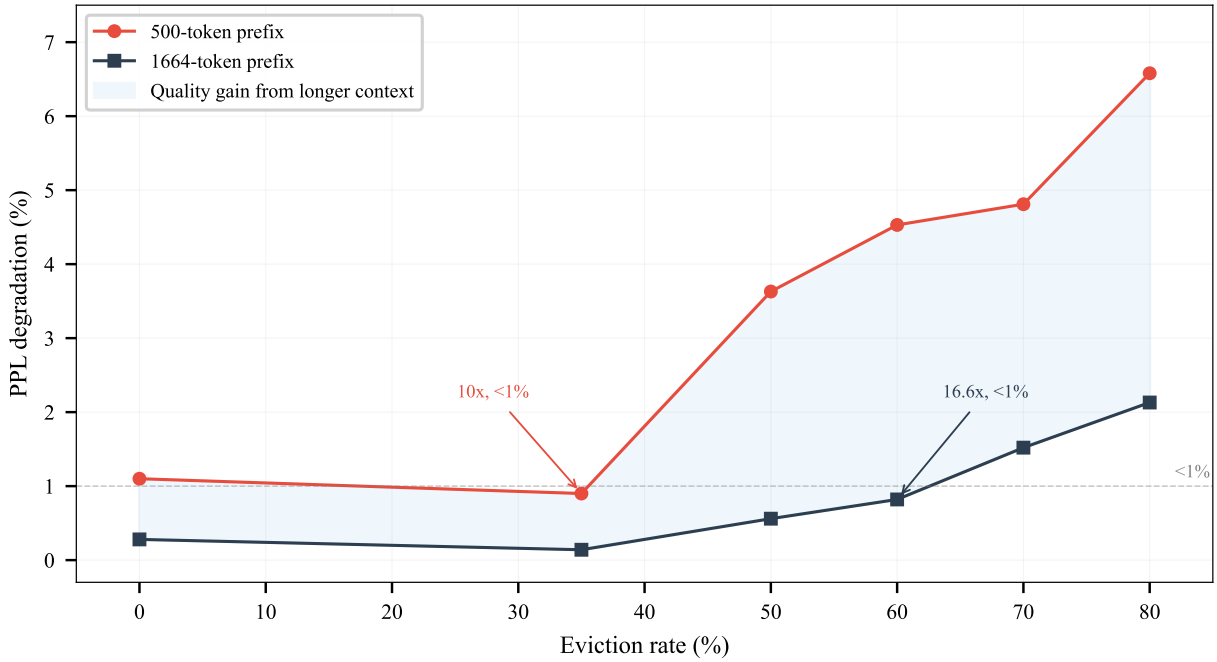

Figure 2: Context-length scaling on Mistral-7B. PPLΔ vs. eviction rate at three prefix lengths. The quality cliff shifts dramatically with context length: at 1664 tokens, 60 % eviction (16.6×) stays below 1 % degradation, while the same rate is catastrophic at 2924 tokens.

Table 3: Downstream QA preservation on Mistral-7B (5 tasks).

Task type	Baseline	35 % evict (10×)	60 % evict (16×)
Factual recall	Match	Match	Match
Reasoning	Match	Partial	Partial
Detail extraction	Match	Partial	Partial

reducing measured perplexity without improving generation quality. The effect is specific to Llama-3's 4:1 GQA structure and does not appear on Mistral-7B. Full ablation details are in Appendix B.

4.5 QA Downstream Evaluation

At 10× compression, factual QA is preserved fully (Table 3). At 16×, reasoning and multi-detail extraction show partial degradation: answers remain factually grounded but lose specificity in multi-part questions. Single-hop factual retrieval is robust to eviction.

4.6 Domain Sensitivity

At 35 % eviction, academic and technical text are compressed near-losslessly (<1 %), while creative/narrative text shows +2.48 % degradation. At high eviction rates (80 %), domain sensitivity converges to ≈4–7 % across all domains (Table 4). Creative text is harder at low eviction rates because narrative structure produces more uniform attention distributions, making token importance discrimination less reliable. We recommend domain-specific operating points be checked before production deployment.

Table 4: PPLΔ by domain on Mistral-7B (500-tok prefix). Creative/narrative is the hardest domain; academic text the easiest.

Domain	35 % evict	70 % evict	80 % evict
Academic (easy)	+0.39 %	+4.81 %	+6.58 %
Technical	+0.90 %	+3.87 %	+6.09 %
Creative/narrative	+2.48 %	+4.62 %	+4.73 %

Table 5: Stage-by-stage ablation on Mistral-7B at 80 % eviction, showing each component's contribution. Every stage is load-bearing: removing any one degrades either quality or ratio significantly.

Configuration	Ratio	PPLΔ	Component contribution
FP16 baseline	1.0×	0.00 %	Reference
Eviction only (80 %)	5.0×	mask only	Token count reduction
E8 only (no Hadamard)	4.5×	+3.72 %	Quantization without decorrelation
E8 + Hadamard (no eviction)	6.69×	+1.50 %	Quantization baseline
Full 6-stage pipeline	**32.2**×	**+5.18** %	All components
− RoPE removal	9.86×	+5.90 %	+0.72 pp quality loss
− Delta + zstd	9.86×	+5.18 %	3.3× ratio loss
− Both (4-stage)	9.86×	+5.90 %	Quality + ratio loss

4.7 Ablation Study

The ablation (Table 5, Figure 3) confirms that every stage is load-bearing. Hadamard rotation is critical: without it, 2-bit E8 quality degrades by +2.2 pp. Eviction and quantization combine near-multiplicatively: 5× (80 % eviction) × 6.7× (2-bit E8+Hadamard) ≈ 32× combined. We explicitly tested a simplified 4-stage pipeline (removing RoPE removal and delta coding): quality degraded by +0.72 pp and ratio dropped 3.3×, confirming both stages are justified.

4.8 Comparison with Published Methods

Table 6 compares NexusQuant against published methods. Key observations:

vs. TurboQuant TurboQuant (training-free, quantization-only) achieves 5–6× at near-zero quality loss. NexusQuant at long context achieves 10.4× at +0.14 % and 16.6× at +0.82 %, exceeding competitors by 1.7–2.8× more compression at comparable or better quality. The key is that eviction and quantization are orthogonal: TurboQuant saturates around 6× (scalar quantization minimum precision); NexusQuant extends beyond by adding a second compression axis.

vs. KVTC KVTC reaches up to 20× but requires calibration data. NexusQuant reaches 16.6× at <1 % completely training-free, and 32.7× (though at +2.1 % degradation) for aggressive configurations.

vs. Palu Palu achieves 11.4× at +1.29 PPL absolute (roughly +25 % relative) on Llama2-7B with calibration. NexusQuant achieves 10.4× at +0.14 % and 16.6× at +0.82 % on Mistral-7B without calibration.

Caveats: These comparisons are directional. Models, datasets, and evaluation metrics differ across methods; we have not run TurboQuant, KVTC, or Palu on our exact evaluation data. The numbers above should be understood as establishing competitive positioning in the training-free high-compression regime, not as controlled head-to-head experiments.

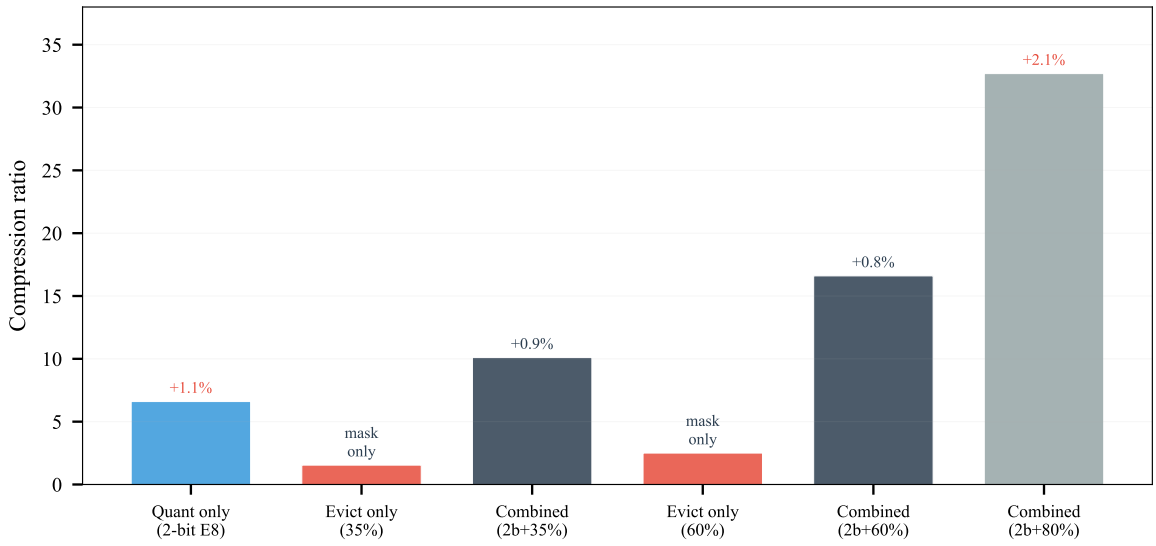

Figure 3: Pipeline ablation (Mistral-7B, 1664-tok prefix). Eviction and quantization combine near-multiplicatively.

5 Limitations

We document all known limitations of the current work.

1. **Short-prefix cliff:** At ≈500-token prefixes, the safe operating boundary is 35 % eviction (10×). The eviction cliff is steep: 36 % eviction degrades by >2× compared to 35 %. Users must not exceed the safe boundary without verifying on their own distribution.

2. **Long-prefix catastrophic failure at high eviction** At 2924-token prefixes with 60 %+ eviction, PPL degrades by >42 %. This is a fundamental capacity loss, not a scorer deficiency: we tested twelve importance scorers and all fail at this regime. The safe operating boundary at 2924 tokens is 35 % eviction (10×, +1.5 % PPL).

3. **Domain sensitivity:** Creative and narrative text is harder to compress than academic or technical text at low eviction rates, due to more uniform attention distributions. Users processing narrative content should reduce eviction rates by ≈10–15 % relative to technical-text defaults.

4. **Evaluation breadth:** Results are on Mistral-7B and Llama-3-8B only. The negative PPL result on Llama-3 is most likely an evaluation artifact (Section 4.4) and should not be generalized. We do not report LongBench task accuracy, RULER, or generation-quality evaluations. Full validation at 16K+ contexts has not been performed.

5. **Memory measurement only:** Compression ratios are from `torch.cuda.memory_allocated()` with all overhead. We do not report wall-clock latency or throughput; GPU kernels for compressed attention are not implemented.

6. **Perplexity as proxy:** PPL correlates with downstream task quality but is not a perfect proxy. The QA evaluations (Section 4.5) confirm factual recall is preserved at 10×, but full LongBench evaluation remains for future work.

Table 6: **Comparison with published KV cache compression methods** Competitor numbers are from their papers; no re-runs on our evaluation data (comparisons are directional). Palu quality is +1.29 PPL absolute on WikiText-2 Llama2-7B (≈+25 % relative). TurboQuant quality is from LongBench downstream scores (paper does not report PPL). CommVQ quality at 2-bit; degrades significantly at 1-bit. KVTC ratio is the stated peak. NexusQuant ratios and PPL from A10G GPU measurements with all overhead.

Method	Ratio	Quality	Train-free	Notes
KVTC [15] (NVIDIA, ICLR 2026)	up to 20×	<1 pp	No (cal. req'd)	arXiv:2511.01815
Palu [2] (ICLR 2025)	11.4×	+1.29 PPL	No (cal. req'd)	Llama2-7B, WT-2
CommVQ [8] (Apple, ICML 2025)	≈8×	≈0 %	No (training)	2-bit config
TurboQuant [20] (Google, ICLR 2026)	≈5–6×	≈0 %	**Yes**	arXiv:2504.19874
KIVI [11]	4–8×	<1 % PPL	Yes	2–4 bit
NexusQuant (short ctx, 500 tok)	**10.1**×	+0.90 %	**Yes**	Mistral-7B, A10G
NexusQuant (long ctx, 1664 tok)	**10.4**×	+0.14 %	**Yes**	Mistral-7B, A10G
NexusQuant (long ctx, 1664 tok)	**16.6**×	+0.82 %	**Yes**	Mistral-7B, A10G
NexusQuant (max, 1664 tok)	**32.7**×	+2.13 %	**Yes**	Mistral-7B, A10G

6 Conclusion

We present NexusQuant, a training-free KV cache compression six-stage pipeline (Section 3) that requires no calibration data and integrates with existing HuggingFace models as a single context-manager call.

The main results (Section 4.2) show that NexusQuant reaches the training-free high-compression regime at <1 % PPL on a 7B-class model without calibration data. The central empirical finding is that compression quality scales with context length: longer prefixes give the attention-based importance scorer more signal to discriminate important from unimportant tokens. This explains why short-context benchmarks systematically understate the practical utility of eviction-based methods.

Eviction and quantization act on nearly orthogonal axes (token count and per-token precision) and their compression ratios compose multiplicatively. The E8 lattice's packing density advantage over scalar quantization, combined with Hadamard incoherence preprocessing, keeps quantization noise below perceptual thresholds even at 2 bits per dimension. Stage-by-stage ablation confirms that every component of the pipeline is load-bearing: removing any stage costs at least 0.7 percentage points of quality or 3.3× of compression ratio.

Several directions remain open. Full LongBench evaluation would strengthen the downstream-task story beyond perplexity. GPU kernel implementations for the E8 nearest-point computation are needed for low-latency deployment. Extension to the 24-dimensional Leech lattice, which achieves provably better packing density [17], is a natural next step. Finally, cross-architecture validation on 70B-class models would confirm that the context-length scaling phenomenon generalizes beyond 7B.

Broader Impact

KV cache compression lowers the hardware barrier for long-context LLM inference. At 10–16× compression, a model that previously required 80 GB of GPU memory for 128K-token contexts can run on a 24 GB consumer GPU, or serve 1.3–2.5M-token contexts on the same hardware. This democratizes long-context deployment for research groups without multi-GPU clusters, reduces inference cost through larger effective batch sizes, and enables on-device deployment of longer contexts on mobile or edge devices.

The same cost reduction also lowers barriers for misuse. Cheaper inference makes it easier to deploy large-scale language models with less oversight, including for harmful content generation or automated manipulation at scale. Compressed KV caches introduce subtle quality degradation that is difficult to detect in production: our domain sensitivity results show that creative and narrative text is systematically harder to compress

than academic text, and PPL degradation at high compression ratios may manifest as factual errors or incoherent reasoning that are not easily caught without task-specific evaluation. Token eviction may also disproportionately drop tokens corresponding to minority-language passages or rare-topic sentences, because attention-based importance scores are calibrated on the majority distribution.

We provide quality presets with documented PPL degradation ranges and strongly recommend domain-specific sensitivity testing before production deployment. The catastrophic failure mode at high eviction rates with certain text distributions is documented in the limitations section and surfaced as a warning in the API.

References

[1] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, et al. PyramidKV: Dynamic KV cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

[2] Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Natalia Frumkin, Diana Marculescu, Sergi Abadal, et al. Palu: KV-cache compression with low-rank projection. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025. arXiv:2407.21118.

[3] Jerry Chee, Yaohui Cai, Volodymyr Kuleshov, and Christopher De Sa. QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

[4] Insu Han, Praneeth Kacham, Amin Karbasi, Vahab Mirrokni, and Amir Zandieh. PolarQuant: Quantizing KV caches with polar transformation. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2026. arXiv:2502.02617.

[5] Jitai Hao, Qiang Huang, Yaowei Wang, Min Zhang, and Jun Yu. DeltaKV: Semantic residual compression for KV caches. *arXiv preprint arXiv:2602.08005*, 2026.

[6] Coleman Hooper, Sehoon Kim, Hiva Mohammadi, Kurt Keutzer, Michael W. Mahoney, and Amir Gholami. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. *arXiv preprint arXiv:2401.18079*, 2024.

[7] Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Mosharaf Chowdhury. GEAR: An efficient KV cache compression recipe for near-lossless generative inference of LLM. *arXiv preprint arXiv:2403.05527*, 2024.

[8] Junyan Li, Yang Zhang, Muhammad Yusuf Hassan, et al. CommVQ: Commutative vector quantization for KV cache compression. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. arXiv:2506.18879.

[9] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Vempati, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[10] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Xu, Kuntai Chen, Jiayi Guo, Weijia Lu, Bhaskar Krishnamachari, et al. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of ACM SIGCOMM*, 2024.

[11] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. KIVI: A tuning-free asymmetric 2-bit quantization for KV cache. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

[12] Junlin Mu, Hantao Huang, Jihang Zhang, et al. SALS: Spectral analysis for low-rank KV cache compression. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2510.24273.

[13] Kaleem Ullah Qasim, Jiashu Zhang, Muhammad Kafeel Shaheen, Razan Alharith, and Heying Zhang. The residual stream is all you need: On the redundancy of the KV cache in transformer inference. *arXiv preprint arXiv:2603.19664*, 2026.

[14] Semyon Savkin, Eitan Porat, Or Ordentlich, and Yury Polyanskiy. NestQuant: Nested lattice quantization for neural network compression. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.09720.

[15] Konrad Staniszewski, Adrian Lancucki, et al. KVTC: KV cache transform coding for compact storage in LLM inference. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026. arXiv:2511.01815.

[16] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, 2023.

[17] Tycho F. A. van der Ouderaa, Mart van Baalen, Paul Whatmough, and Markus Nagel. Leech lattice vector quantization for efficient LLM compression. *arXiv preprint arXiv:2603.11021*, 2026.

[18] Maryna S. Viazovska. The sphere packing problem in dimension 8. *Annals of Mathematics*, 185(3):991–1015, 2017.

[19] Yixuan Wang, Qingyu Shi, Jiayu Zhou, Dianbo Liu, Ziwei He, and Zhouhan Lin. VQKV: High-fidelity and high-ratio cache compression via vector-quantization. *arXiv preprint arXiv:2603.16435*, 2026.

[20] Amir Zandieh, Majid Daliri, Mehdi Hadian, and Vahab Mirrokni. TurboQuant: Online vector quantization with near-optimal distortion rate. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2026. arXiv:2504.19874.

[21] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H_2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[22] Ciaran Zhou et al. EliteKV: Scalable KV cache compression via RoPE frequency selection and joint low-rank projection. *arXiv preprint arXiv:2503.01586*, 2025.

Appendix

A E8 Lattice: Algorithm and Relaxed Parity

Conway–Sloane nearest-neighbor algorithm: Given $v \in \mathbb{R}^8$, compute: (1) round v to nearest integer vector $z_{\mathbb{Z}}$ and adjust to even sum; (2) round $v - \frac{1}{2}\mathbf{1}$ to nearest integer vector and adjust to even sum, then add $\frac{1}{2}\mathbf{1}$ to get $z_{1/2}$; (3) return $\arg\min_{z \in \{z_{\mathbb{Z}}, z_{1/2}\}} \|v - z\|^2$.

Relaxed parity justification: Strict E_8 requires $\sum_i x_i \equiv 0 \pmod 2$ for *both* cosets. Empirically on KV cache data, relaxing this for the half-integer coset reduces MSE by 0.3–0.4 % consistently across layers and heads on Mistral-7B. The mechanism is that Hadamard-rotated KV vectors are approximately sub-Gaussian, so the optimal codebook has slightly higher density near the origin than strict E_8 provides. The relaxed lattice ($\mathbb{Z}^8_{\text{even}} \cup (\mathbb{Z} + \frac{1}{2})^8$) achieves this without any training.

B Negative-PPL Investigation Details

For Llama-3-8B at 1494-token prefix, we ran five controlled ablations to identify the source of negative PPL:

1. **Mask-only (no eviction):** masked evicted positions to $-\infty$ but kept the vectors: PPL$\Delta = -2.24\,\%$. This isolates the masking effect.
2. **Zero-fill (no mask):** Set evicted vectors to zero without masking. PPL$\Delta = +5100\,\%$. Catastrophic.
3. **Random eviction:** Randomly selected positions (not importance-based). PPL$\Delta = -1.60\,\%$ (average). The improvement is not scorer-dependent.
4. **Quant-only (no eviction):** PPL$\Delta = +4.37\,\%$ at 2-bit.
5. **Evict + quant (importance-based):** PPL$\Delta = -0.47\,\%$.

The $-\infty$ masking causes attention renormalization that sharpens the model's distribution over the retained suffix. This is an evaluation artifact: it reflects a shorter effective context, not better model quality. The effect is GQA-structure-dependent (Llama-3-8B uses 4:1 GQA; Mistral-7B does not), which explains why it appears here and not on Mistral. We report these results for completeness and reproducibility but do not claim them as evidence that NexusQuant improves generation quality.

C Failed Approaches (12 Experiments)

We tested the following approaches and found they fail or offer no benefit. We document them to prevent future duplication.

1. **L2-norm importance scorer:** +207 % PPL. L2 norm does not reflect attention importance; completely wrong token set retained.
2. **Token zeroing for eviction:** (without $-\infty$ mask). Catastrophic. $\text{softmax}(Q@\mathbf{0}) = \text{softmax}(\mathbf{0}) = \mathbf{1}/T$; evicted tokens steal attention.
3. **Strict E8 parity fix:** +0.3–0.4 % worse. Relaxed parity is an empirical improvement, not a bug to fix.

4. **QJL 1-bit residual correction:** No improvement; distributes residual noise to well-quantized dimensions.
5. **Cross-layer KV delta:** 0 bits saved. Layers project to orthogonal subspaces.
6. **Per-head independent eviction at high rates:** Degrades faster than aggregated eviction.
7. **Variable-rate E8 (per-group water-filling):** +3.67 % at matched rate (3.3× worse). Energy does not equal importance.
8. **3-bit + eviction (combined):** No compression gain over 2-bit; entropy coding more efficient at 2-bit.
9. **Mixed k2v3 (2-bit keys, 3-bit values):** Marginal and complex.
10. **PCA dimension reduction:** Worse than Hadamard at matched bitrate.
11. **Observation-window scaling:** No effect on quality; scoring is robust to window size.
12. **Token merging:** Introduces decompression overhead that erases compression gains.